UNIVERSIDAD
Panamericana

PROGRAMACIÓN AVANZADA

Sesión 01

Presentación del Curso

✎ NOTAS DEL PROFESOR

Universidad Panamericana
Dr. Miguel Alcaraz Rivera • malcaraz@up.edu.mx

2026

Índice

Objetivos de la sesión

1. Presentar al profesor, el curso y sus expectativas.
2. Transmitir el syllabus, la bibliografía y la forma de calificación.
3. Conocer al grupo: niveles de experiencia, expectativas y emociones iniciales frente a la programación.
4. Establecer desde el primer día una cultura de participación activa.
5. Medir la velocidad de escritura como diagnóstico inicial (10FastFingers).

> Esta clase es más de clima y contrato pedagógico que de contenido técnico. El objetivo principal es que los alumnos salgan con ganas de volver. Sé accesible, muestra entusiasmo genuino por el tema y valida que la programación puede ser difícil — y que eso está bien. Evita el discurso de "este curso es muy difícil, muchos van a reprobar": genera ansiedad sin ningún beneficio pedagógico.

Duración y materiales

Actividad	Tiempo estimado
Presentación del profesor y el curso	10 min
Syllabus y calificación	15 min
Bibliografía y recursos	10 min
Padlet — meme de programar	15 min
Lluvia de ideas: ¿buen programador?	20 min
10FastFingers	10 min
Cierre y vista al siguiente clase	5 min
Total	**85 min**

Materiales necesarios:

- Proyector con acceso a internet
- Padlet creado antes de la clase (ver instrucciones en §3)
- Los alumnos con laptop, tablet o smartphone para acceder al Padlet y a 10FastFingers

Secuencia detallada

• Presentación del profesor y del curso (10 min)

Comienza con una breve presentación personal: quién eres, en qué trabajas o has trabajado, qué te apasiona de C++ y la programación. No más de 3 minutos.

Luego una pregunta de sondeo rápida al grupo (manos levantadas):

- ¿Cuántos han programado antes en algún lenguaje?
- ¿Cuántos han visto C o C++ anteriormente?
- ¿Cuántos usan regularmente herramientas de IA para programar?

> El sondeo de IA es especialmente revelador en grupos actuales. Si la mayoría usa Copilot o ChatGPT, ajusta el discurso del curso: enfatiza que entender la memoria y los apuntadores es exactamente lo que distingue a quien puede *evaluar* el código que genera la IA del que solo puede copiarlo.

• Syllabus y calificación (15 min)

Presenta el syllabus proyectado. Detente especialmente en la estructura de calificación: 1/3 tareas, 1/3 asistencia y participación, 1/3 examen.

> El 33 % de asistencia y participación no es arbitrario. La programación se aprende haciendo y discutiendo, no leyendo. Un alumno presente y activo que no llega al 70 en el examen tiene más probabilidad de resolver problemas reales que uno que memorizó para el examen pero nunca participó. Explica esto explícitamente: muchos alumnos no están acostumbrados a que la participación valga tanto.
>
> Define claramente qué es "participación": hacer preguntas, responder, proponer ideas, ayudar a un compañero frente al grupo. No es asistencia silenciosa.

Abre espacio para preguntas sobre el reglamento. Es mejor aclarar dudas sobre entregas, fechas y política de retrasos en la primera clase.

• Bibliografía y recursos (10 min)

Proyecta cppreference.com y learncpp.com. Muestra rápidamente cómo están organizados. No es necesario entrar en detalle — la intención es que sepan que existen y que los usarán durante el semestre.

> Muchos alumnos jamás han consultado documentación técnica formal. Decirles "lean el libro" sin enseñarles a navegarlo es ineficaz. En la sesión 2 harás una demostración en vivo de cppreference — aquí solo siembra la idea.

• Padlet — ¿Qué emoción te produce programar? (15 min)

Configuración previa (hacer antes de la clase):

1. Crear cuenta en `https://padlet.com` si no tienes una.
2. Nuevo padlet → formato **Muro** (Wall).
3. Título: "¿Qué emoción te produce programar?"
4. Activar opción de comentarios si deseas discusión.

5. Copiar el enlace o generar el QR code (botón Share).
6. Proyectar el QR al inicio de la actividad.

Instrucción al grupo:

> "Busca un meme, GIF o imagen que represente cómo te sientes cuando programas — o cómo imaginas que te vas a sentir. Agrégalo al Padlet con tu nombre. Tienes 8 minutos."

Dedica los últimos 7 minutos a revisar los posts con todo el grupo. Comenta algunos, genera risa, normaliza la frustración y el caos que aparezcan.

Tiene tres funciones simultáneas: rompe el hielo, da información diagnóstica sobre la actitud del grupo frente a la programación, y establece que este curso va a tener un tono humano. Los alumnos que suben memes de frustración total son frecuentemente los más comprometidos al final del semestre — solo necesitaban permiso para expresarlo.

Guarda el Padlet. Es un documento vivo: puedes mostrarlos al final del semestre y pedir a los alumnos que comenten si su meme inicial todavía los representa.

• Lluvia de ideas: ¿Qué se necesita para ser un buen programador hoy? (20 min)

Escribe la pregunta en el pizarrón tal cual. Sin definirla, sin restringirla.
Dinámica:

1. 3 minutos de reflexión individual en silencio.
2. 10 minutos de lluvia de ideas abierta: cualquiera habla, tú escribes en el pizarrón sin filtrar ni jerarquizar.
3. 7 minutos de discusión: agrupa las ideas que aparezcan en categorías naturales (técnicas, actitudinales, sociales, etc.).

"Saber muchos lenguajes" — Guía hacia: la profundidad en uno aporta más que la superficialidad en muchos. C++ enseña conceptos que aplican a todos los lenguajes.

"Usar IA / Copilot" — Valida: es una herramienta real. Agrega: para evaluar lo que genera la IA necesitas entender qué está haciendo. Un programador que no entiende apuntadores no puede juzgar si el código generado gestiona bien la memoria.

"Paciencia / no rendirse" — Valida completamente. Es real y está respaldado por la literatura: la perseverancia predice el éxito en programación más que la aptitud inicial.

"Matemáticas" — Matiza: para programación general, no es un requisito estricto. Para ciertos campos (gráficos, ML, criptografía, física) sí importa más. En este curso las matemáticas que necesitarás son mínimas.

"Buscar en Google / Stack Overflow" — Valida: es una habilidad real. Saber formular una búsqueda efectiva es parte del oficio.

Un estudio de la Universidad de Washington (2019) siguió a estudiantes de CS durante 4 años y encontró que la variable más predictiva de completar la carrera no fue la nota de entrada ni el conocimiento previo, sino la **mentalidad de crecimiento** (growth mindset): creer que la inteligencia y la habilidad se desarrollan con esfuerzo.
Referencia: Cutts et al. (2019). "Predict or Pre-empt: How to respond to poor performance in early CS courses". *ICER 2019*.

- ## 10FastFingers — Diagnóstico de mecanografía (10 min)

URL: `https://10fastfingers.com/typing-test/spanish`
Instrucción al grupo:

> "Abre el sitio, elige el modo en español y haz el test de 1 minuto. Anota tus palabras por minuto y tu precisión en tu hoja de notas. Nadie tiene que mostrar su resultado si no quiere."

No es un criterio de calificación. Su valor es diagnóstico y motivacional:

- Diagnostica cuánto tiempo perderá el alumno en actividades simples de escritura durante el semestre.
- Abre la conversación sobre herramientas y hábitos: atajos de teclado, teclado mecánico, práctica deliberada.
- Permite una comparación al final del semestre: los alumnos suelen mejorar 15–25 PPM solo por la práctica acumulada de escribir código.

Si quieres hacer seguimiento, pide voluntarios que compartan su PPM inicial en el Padlet. Puedes repetir el ejercicio al final del semestre.

Comparte con el grupo algunos datos de contexto (ver notas del alumno, §4.3). La idea no es presionar sino motivar: la mecanografía mejora dramáticamente con práctica deliberada de 10–15 minutos al día.

- ## Cierre y vista a la siguiente clase (5 min)

- Resume en una frase qué viene en la sesión 2: "La próxima clase escribimos y ejecutamos nuestro primer programa C++."
- Recuerda el requisito técnico: `g++` instalado o acceso a Google Cloud Shell.

- Deja abierta la pregunta de la lluvia de ideas: "Vayan pensando en si añadirían algo a la lista que armamos hoy."

Las partes del computador — Facilitación de la analogía del escritorio

Preséntala después de la lluvia de ideas o antes del Padlet, según el ritmo del grupo. Si la discusión sobre "qué necesita un buen programador" derivó hacia el hardware o el funcionamiento interno, es el momento perfecto para hacer la transición.

• Cómo facilitar la analogía

Dibuja en el pizarrón un escritorio desde arriba (vista de planta). Añade los elementos uno por uno mientras haces preguntas al grupo:

1. Dibuja al usuario sentado. **"¿Quién hace el trabajo real?"** → CPU.
2. Dibuja papeles sobre el escritorio. **"¿Dónde están las cosas que usas *ahorita*?"** → RAM.
3. Dibuja cajones. **"¿Dónde guardas lo que no estás usando?"** → Almacenamiento (disco / SSD).
4. Dibuja las manos moviéndose. **"¿Cómo llegan los datos de los cajones al escritorio?"** → Bus de datos.
5. Menciona las reglas de la oficina. **"¿Quién decide quién puede usar el escritorio y cuánto espacio tiene cada quien?"** → Sistema operativo.

Cierra con la pregunta: **"¿Qué pasa si llenamos el escritorio?"** → introduce la idea de memoria virtual y por qué la RAM limitada importa.

• Tabla de referencia

Parte	Analogía	Punto clave
CPU	Usuario sentado	Procesa instrucciones en orden
RAM	Espacio sobre el escritorio	Rápida, volátil, limitada
Disco / SSD	Cajones	Persistente, mucho más lento
Bus de datos	Manos del usuario	Transporta entre partes
Sistema operativo	Reglas de la oficina	Administra recursos
Programa	Tarea sobre el escritorio	Datos + instrucciones activos

• La tabla de latencias (dato para compartir)

Muestra la tabla de perspectiva de velocidades. Pide al grupo que adivine las proporciones antes de revelarlas. Suelen sorprender: la diferencia entre RAM y disco mecánico (1 año vs 6 minutos en escala humana) siempre impacta.

La tabla de latencias de Jeff Dean es una de las más citadas en la industria. No se espera que los alumnos la memoricen; el objetivo es desarrollar intuición sobre por qué el acceso a memoria importa en el diseño de programas. Esto resuena directamente cuando lleguemos a apuntadores y memoria dinámica: cada `new` tiene un costo real.
Referencia: `https://gist.github.com/jboner/2841832`

- **Conexión explícita con el resto del curso**

Termina la sección señalando el hilo conductor: **todo este curso es sobre entender y controlar ese escritorio**. Cuando veamos apuntadores estaremos hablando literalmente de celdas numeradas de la RAM. Cuando veamos `new`/`delete`, estaremos decidiendo cuándo poner y quitar cosas del escritorio sin que nadie lo haga por nosotros.

Adaptaciones según el perfil del grupo

Dedica más tiempo al Padlet y a la lluvia de ideas. La ansiedad ante lo desconocido es real; nombrarla y normalizarla es más efectivo que minimizarla. Puedes agregar una ronda de presentaciones personales al inicio.

El sondeo inicial revelará esto. Ajusta el discurso: C++ no es "otro lenguaje más" — es un cambio de modelo mental. La gestión explícita de memoria y la falta de garbage collector son el salto conceptual más importante. Puedes hacer la pregunta de lluvia de ideas más específica: "¿Qué se necesita para *dominar* C++ viniendo de Python/JavaScript?"

Recursos adicionales para el profesor

- **Padlet (guía rápida):** `https://padlet.com/support`
- **10FastFingers:** `https://10fastfingers.com`
- **Índice TIOBE C++:** `https://www.tiobe.com/tiobe-index/cplusplus/`
- **Growth mindset en CS:** Dweck, C. (2006). *Mindset: The New Psychology of Success*. Random House.
- **C++ Core Guidelines (referencia de calidad de código):** `https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines`

Material de la sesión

- **Notas del alumno:** `notas_alumno/pdf/sesion01_presentacion.pdf`

- **Siguiente sesión:** `sesiones/sesion02_introduccion_cpp.ipynb`